

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

Студент: Николаева Ангелина

Борисовна

Студенческий билет:

1032253612

Группа: НКАбд-04-25

МОСКВА

2026 г

Содержание

1. Цель работы	4
2. Выполнение лабораторной работы.....	5
2.1. Символьные и численные данные в NASM.....	5
2.2. Выполнение арифметических операций в NASM.....	10
2.3. Ответы на контрольные вопросы.....	13
2.4. Задание для самостоятельной работы	14
3. Выводы	16

Список иллюстраций

1.	Создание нового каталога	9
2.	Сохранение новой программы	10
3.	Запуск изначальной программы	10
4.	Измененная программа.....	11
5.	Запуск измененной программы.....	11
6.	Вторая программа	12
7.	Вывод второй программы	12
8.	Вывод измененной второй программы.....	13
9.	Замена функции вывода во второй программе	13
10.	Третья программа	14
11.	Запуск третьей программы.....	14
12.	Изменение третьей программы	15
13.	Запуск измененной третьей программы	15
14.	Программа для подсчета варианта	16
15.	Запуск программы для подсчета варианта	16
16.	Запуск и проверка программы.....	18

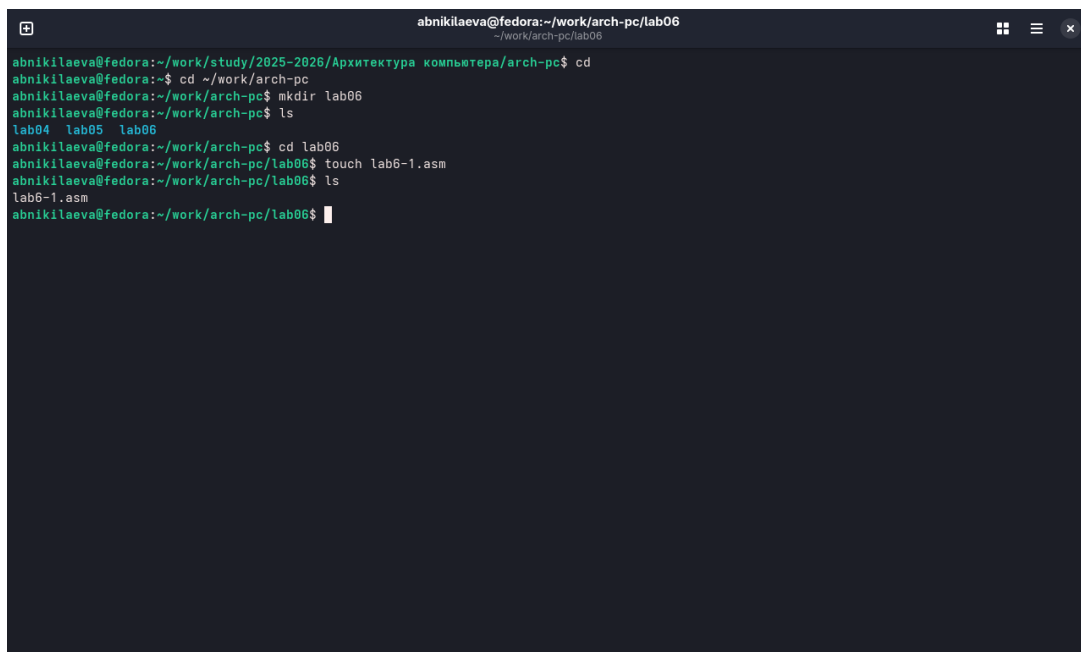
1. Цель работы

Цель данной лабораторной работы - освоение арифметических инструкций языка ассемблера NASM.

2. Выполнение лабораторной работы

2.1. Символьные и численные данные в NASM

Создаю каталог для программ лабораторной работы №6 и перехожу в него, создаю там файл (рис. 2.1).



```
abnikilaeva@fedora: ~/work/arch-pc/lab06
abnikilaeva@fedora:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ cd
abnikilaeva@fedora:~$ cd ~/work/arch-pc
abnikilaeva@fedora:~/work/arch-pc$ mkdir lab06
abnikilaeva@fedora:~/work/arch-pc$ ls
lab04  lab05  lab06
abnikilaeva@fedora:~/work/arch-pc$ cd lab06
abnikilaeva@fedora:~/work/arch-pc/lab06$ touch lab6-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ ls
lab6-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.1: Создание нового каталога

В созданном файле ввожу программу из листинга (рис. 2.2).



```
abnikilaeva@fedora: ~/work/arch-pc/lab06 — nano lab6-1.asm
GNU nano 8.5 lab6-1.asm Изменен
#include 'in_out.asm'

SECTION .bss
buf1: RESB 80

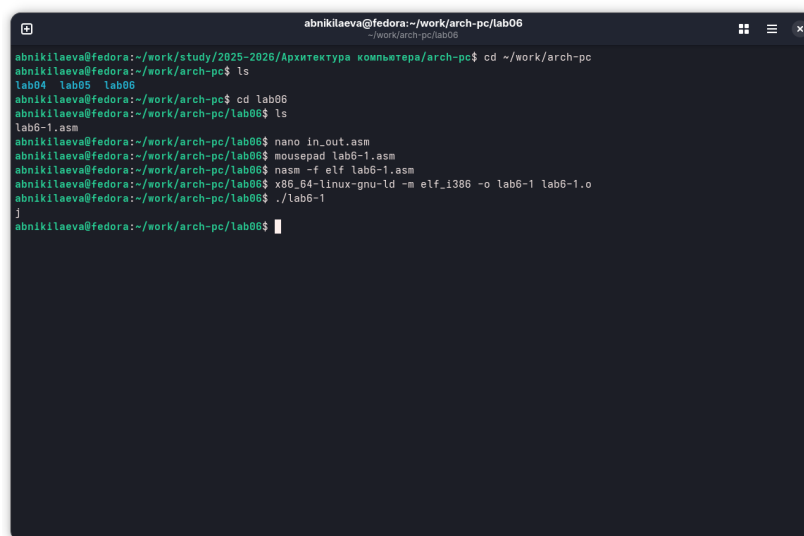
SECTION .text
GLOBAL _start

_start:
    mov eax, '6'
    mov ebx, '4'
    add eax, ebx
    mov [buf1], eax
    mov eax, buf1
    call sprintf
    call quit

Сохранить изменённый буфер?
Y Да
N Нет
[Ctrl] O Отмена
```

Рис. 2.2: Сохранение новой программы

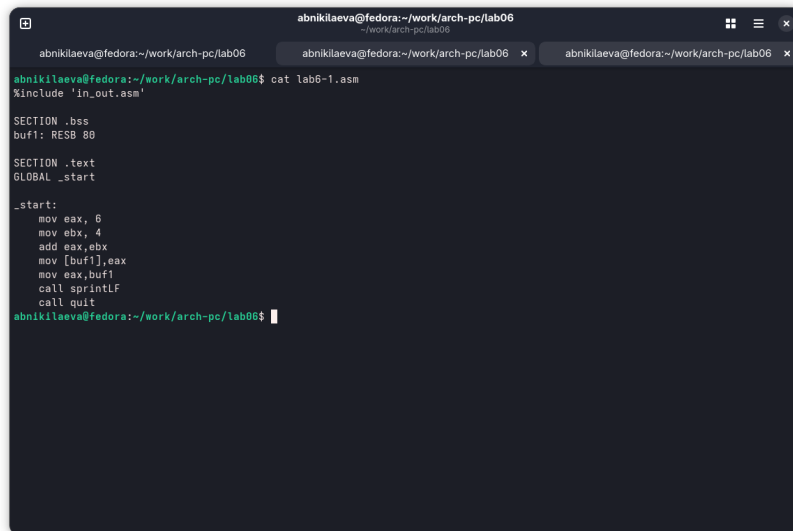
Создаю исполняемый файл и запускаю его, вывод программы отличается от предполагаемого изначально, ибо коды символов в сумме дают символ j по таблице ASCII.



```
abnikilaeva@fedora: ~/work/arch-pc/lab06
abnikilaeva@fedora: ~/work/study/2025-2026/Архитектура компьютера/arch-pc$ cd ~/work/arch-pc
abnikilaeva@fedora: ~/work/arch-pc$ ls
lab04 lab05 lab06
abnikilaeva@fedora: ~/work/arch-pc$ cd lab06
abnikilaeva@fedora: ~/work/arch-pc/lab06$ ls
lab6-1.asm
abnikilaeva@fedora: ~/work/arch-pc/lab06$ nano in_out.asm
abnikilaeva@fedora: ~/work/arch-pc/lab06$ mousepad lab6-1.asm
abnikilaeva@fedora: ~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
abnikilaeva@fedora: ~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_i386 -o lab6-1 lab6-1.o
abnikilaeva@fedora: ~/work/arch-pc/lab06$ ./lab6-1
j
abnikilaeva@fedora: ~/work/arch-pc/lab06$
```

Рис. 2.3: Запуск изначальной программы

Изменяю текст изначальной программы, убрав кавычки (рис. 2.4).

A terminal window with a dark background and light green text. The window title is 'abnikilaeva@fedora:~/work/arch-pc/lab06'. The prompt is 'abnikilaeva@fedora:~/work/arch-pc/lab06\$'. The command 'cat lab6-1.asm' has been executed, displaying the following assembly code:

```
abnikilaeva@fedora:~/work/arch-pc/lab06$ cat lab6-1.asm
%include 'in_out.asm'

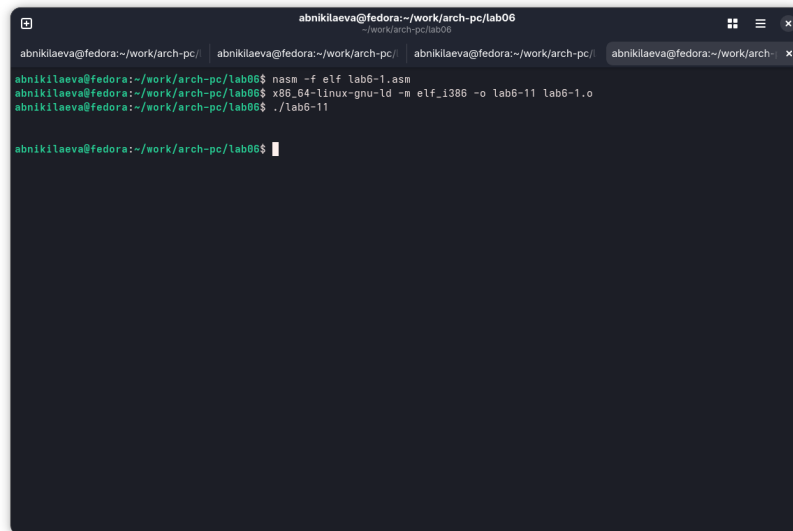
SECTION .bss
buf1: RESB 80

SECTION .text
GLOBAL _start

_start:
    mov eax, 6
    mov ebx, 4
    add eax, ebx
    mov [buf1], eax
    mov eax, buf1
    call sprintf
    call quit
```

Рис. 2.4: Измененная программа

На этот раз программа выдала пустую строку, это связано с тем, что символ 10 означает переход на новую строку (рис. 2.5).

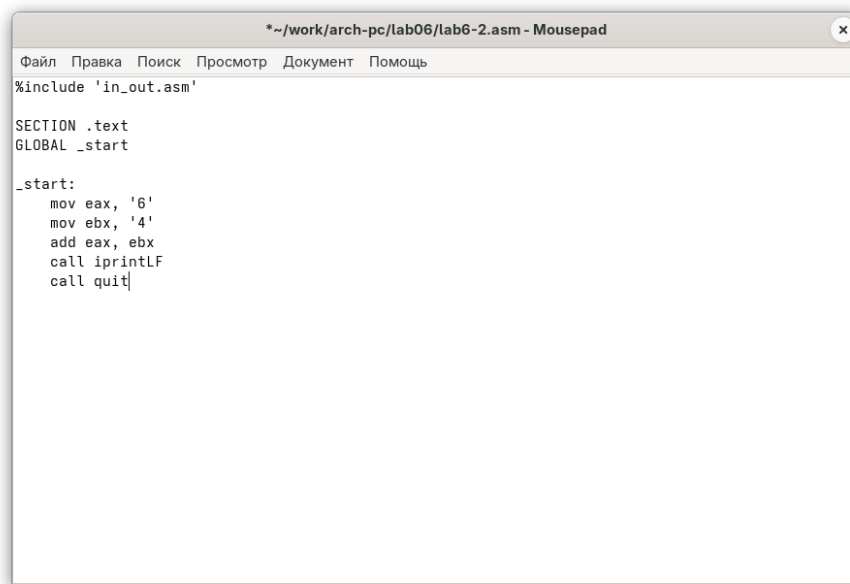
A terminal window with a dark background and light green text. The window title is 'abnikilaeva@fedora:~/work/arch-pc/lab06'. The prompt is 'abnikilaeva@fedora:~/work/arch-pc/lab06\$'. The command 'nasm -f elf lab6-1.asm' has been executed, followed by 'x86_64-linux-gnu-ld -m elf_x86_64 -o lab6-1.o lab6-1.o', and finally './lab6-1'. The output is a single empty line.

```
abnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_x86_64 -o lab6-1.o lab6-1.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-1

```

Рис. 2.5: Запуск измененной программы

Создаю новый файл для будущей программы и записываю в нее код из листинга (рис. 2.6).



```
*~/work/arch-pc/lab06/lab6-2.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

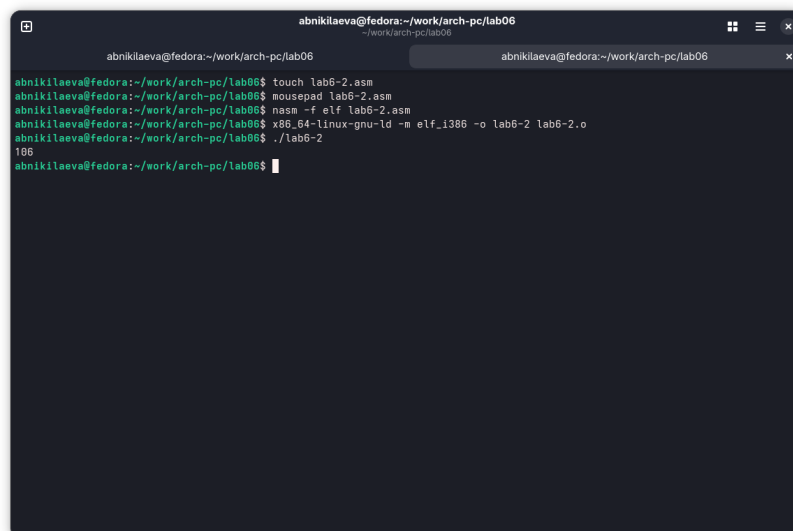
%include 'in_out.asm'

SECTION .text
GLOBAL _start

_start:
    mov eax, '6'
    mov ebx, '4'
    add eax, ebx
    call iprintLF
    call quit
```

Рис. 2.6: Вторая программа

Создаю исполняемый файл и запускаю его, теперь отображается результат 106, программа, как и в первый раз, сложила коды символов, но вывела само число, а не его символ, благодаря замене функции вывода на `iprintLF` (рис. 2.7).

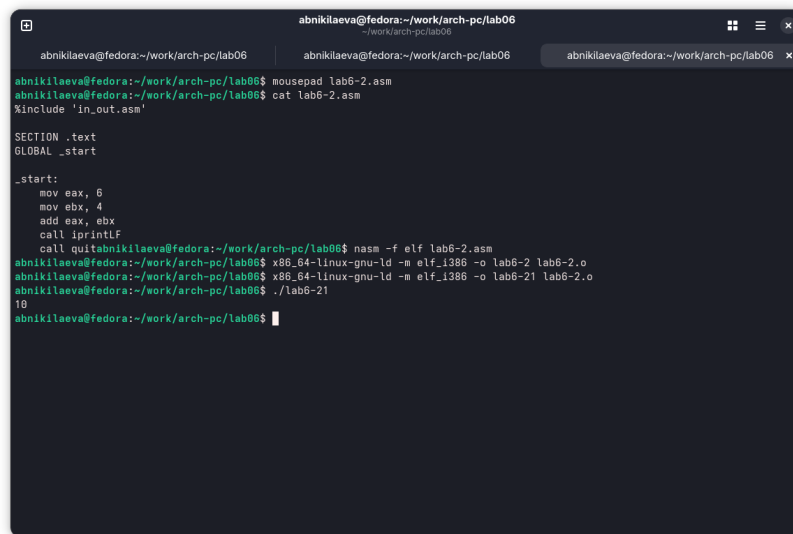


```
abnikilaeva@fedora:~/work/arch-pc/lab06
~/work/arch-pc/lab06
abnikilaeva@fedora:~/work/arch-pc/lab06
abnikilaeva@fedora:~/work/arch-pc/lab06$ touch lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ mousepad lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-2 lab6-2.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-2
106
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.7: Вывод второй программы

Убрав кавычки в программе, я снова ее запускаю и получаю предполагаемый

изначально результат. (рис. 2.8).



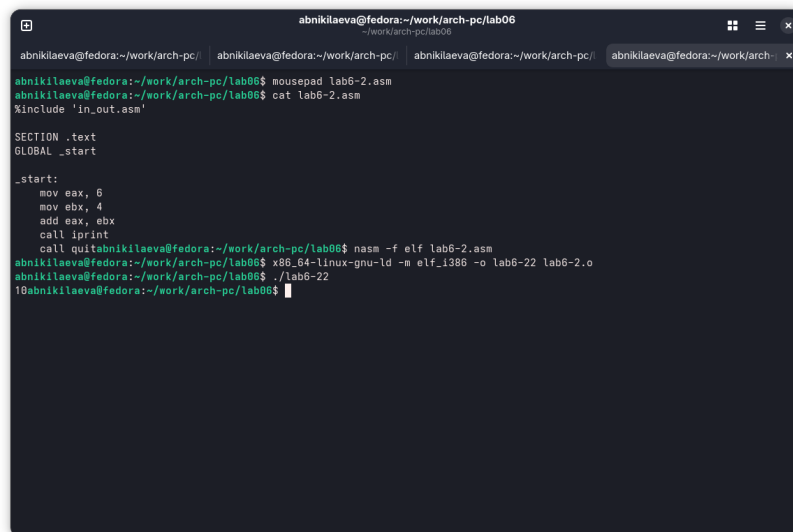
```
abnikilaeva@fedora:~/work/arch-pc/lab06
abnikilaeva@fedora:~/work/arch-pc/lab06 mousepad lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ cat lab6-2.asm
%include 'in_out.asm'

SECTION .text
GLOBAL _start

_start:
    mov eax, 6
    mov ebx, 4
    add eax, ebx
    call iprintlnf
    call quitabnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-2 lab6-2.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-21 lab6-2.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-21
10
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.8: Вывод измененной второй программы

Заменяв функцию вывода на `iprintln`, я получаю тот же результат, но без переноса строки (рис. 2.9).



```
abnikilaeva@fedora:~/work/arch-pc/lab06
abnikilaeva@fedora:~/work/arch-pc/lab06 mousepad lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ cat lab6-2.asm
%include 'in_out.asm'

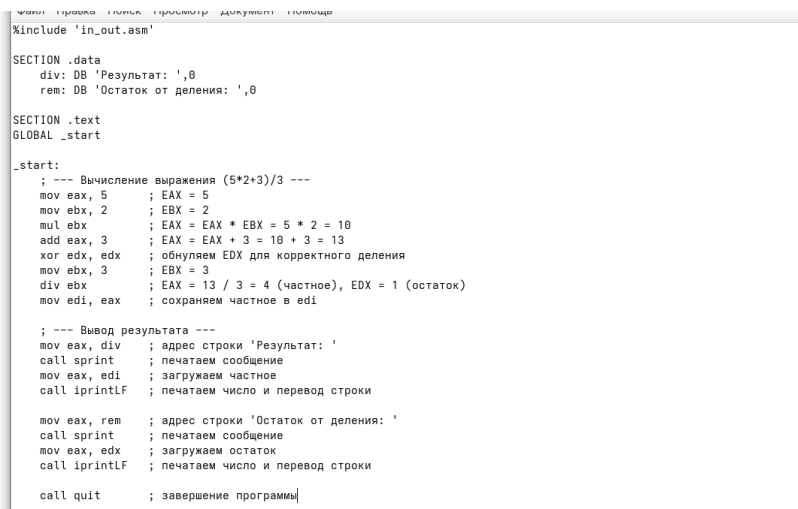
SECTION .text
GLOBAL _start

_start:
    mov eax, 6
    mov ebx, 4
    add eax, ebx
    call iprintln
    call quitabnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-2 lab6-2.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-22 lab6-2.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-22
10abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.9: Замена функции вывода во второй программе

2.2. Выполнение арифметических операций в NASM

Создаю новый файл и копирую в него содержимое листинга (рис. 2.10).



```
%include 'in_out.asm'

SECTION .data
    div: DB 'Результат: ',0
    rem: DB 'Остаток от деления: ',0

SECTION .text
    GLOBAL _start

_start:
    ; --- Вычисление выражения (5*2+3)/3 ---
    mov eax, 5      ; EAX = 5
    mov ebx, 2      ; EBX = 2
    mul ebx         ; EAX = EAX * EBX = 5 * 2 = 10
    add eax, 3      ; EAX = EAX + 3 = 10 + 3 = 13
    xor edx, edx    ; обнуляем EDX для корректного деления
    mov ebx, 3      ; EBX = 3
    div ebx         ; EAX = 13 / 3 = 4 (частное), EDX = 1 (остаток)
    mov edi, eax    ; сохраняем частное в edi

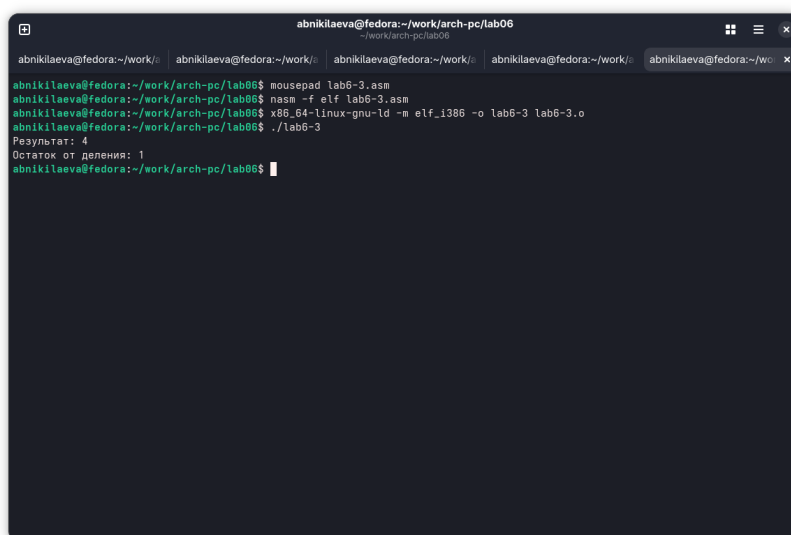
    ; --- Вывод результата ---
    mov eax, div    ; адрес строки 'Результат: '
    call sprint     ; печатаем сообщение
    mov eax, edi    ; загружаем частное
    call iprintlf   ; печатаем число и перевод строки

    mov eax, rem    ; адрес строки 'Остаток от деления: '
    call sprint     ; печатаем сообщение
    mov eax, edx    ; загружаем остаток
    call iprintlf   ; печатаем число и перевод строки

    call quit      ; завершение программы
```

Рис. 2.10: Третья программа

Программа выполняет арифметические вычисления, на вывод идет результи-рующее выражения и его остаток от деления (рис. 2.11).



```
abnikilaeva@fedora: ~/work/arch-pc/lab06
abnikilaeva@fedora:~/work/ abnikilaeva@fedora:~/work/ abnikilaeva@fedora:~/work/ abnikilaeva@fedora:~/wo
abnikilaeva@fedora:~/work/arch-pc/lab06$ mousepad lab6-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-3 lab6-3.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.11: Запуск третьей программы

Заменяю переменные в программе для выражения $f(x) = (4*6+2)/5$ (рис. 2.12).

```
~/work/arch-pc/lab06/lab6-3.asm - mousepad
Файл Правка Поиск Просмотр Документ Помощь
%include 'in_out.asm'

SECTION .data
    div: DB 'Результат: ',0
    rem: DB 'Остаток от деления: ',0

SECTION .text
GLOBAL _start

_start:
    ; --- Вычисление выражения (5*2+3)/3 ---
    mov eax, 4      ; EAX = 5
    mov ebx, 6      ; EBX = 2
    mul ebx         ; EAX = EAX * EBX = 5 * 2 = 10
    add eax, 2      ; EAX = EAX + 3 = 10 + 3 = 13
    xor edx, edx    ; обнуляем EDX для корректного деления
    mov ebx, 5      ; EBX = 3
    div ebx         ; EAX = 13 / 3 = 4 (частное), EDX = 1 (остаток)
    mov edi, eax    ; сохраняем частное в edi

    ; --- Вывод результата ---
    mov eax, div    ; адрес строки 'Результат: '
    call sprint     ; печатаем сообщение
    mov eax, edi    ; загружаем частное
    call iprintf    ; печатаем число и перевод строки

    mov eax, rem    ; адрес строки 'Остаток от деления: '
    call sprint     ; печатаем сообщение
    mov eax, edx    ; загружаем остаток
    call iprintf    ; печатаем число и перевод строки

    call quit      ; завершение программы
```

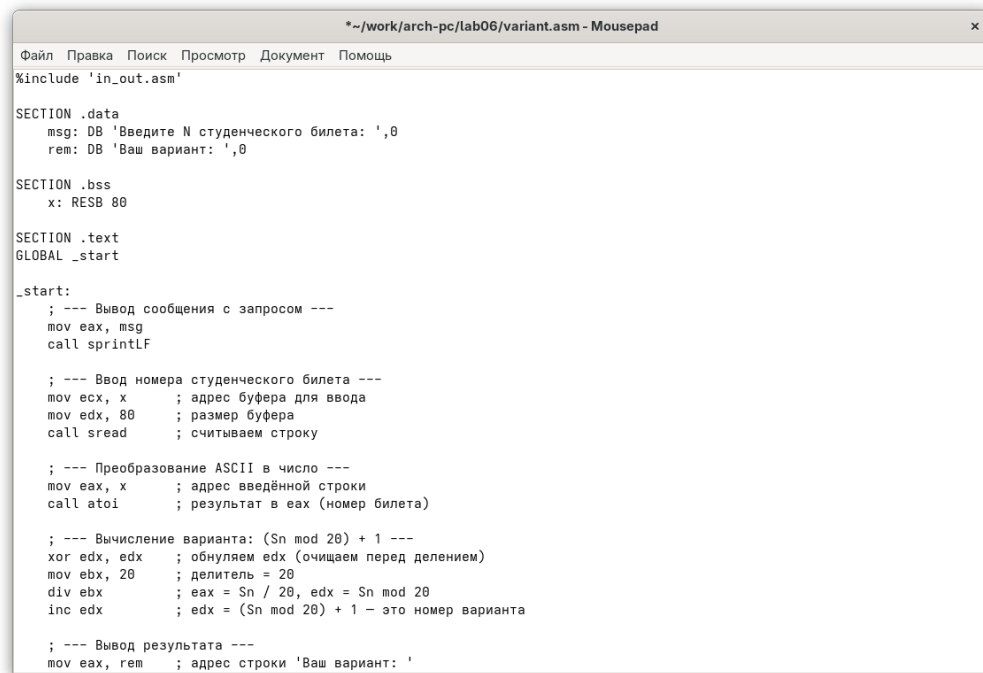
Рис. 2.12: Изменение третьей программы Запуск

программы дает корректный результат (рис. 2.13).

```
abnikilaeva@fedora:~/work/arch-pc/lab06
abnikilaeva@fedora:~/v abnikilaeva@fedora:~/v abnikilaeva@fedora:~/v abnikilaeva@fedora:~/v abnikilaeva@fedora:~/v
abnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o lab6-31 lab6-3.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-31
Результат: 5
Остаток от деления: 1
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.13: Запуск измененной третьей программы Создаю новый

файл и помещаю текст из листинга (рис. 2.14).



```
File  Правка  Поиск  Просмотр  Документ  Помощь
%include 'in_out.asm'

SECTION .data
msg: DB 'Введите N студенческого билета: ',0
rem: DB 'Ваш вариант: ',0

SECTION .bss
x: RESB 80

SECTION .text
GLOBAL _start

_start:
; --- Вывод сообщения с запросом ---
mov eax, msg
call sprintf

; --- Ввод номера студенческого билета ---
mov ecx, x          ; адрес буфера для ввода
mov edx, 80         ; размер буфера
call sread          ; считываем строку

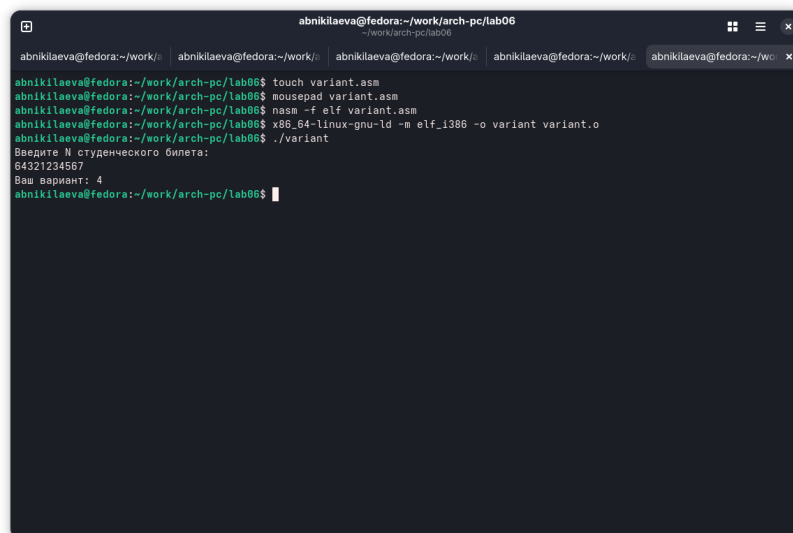
; --- Преобразование ASCII в число ---
mov eax, x          ; адрес введённой строки
call atoi           ; результат в eax (номер билета)

; --- Вычисление варианта: (Sn mod 20) + 1 ---
xor edx, edx        ; обнуляем edx (очищаем перед делением)
mov ebx, 20         ; делитель = 20
div ebx             ; eax = Sn / 20, edx = Sn mod 20
inc edx             ; edx = (Sn mod 20) + 1 — это номер варианта

; --- Вывод результата ---
mov eax, rem        ; адрес строки 'Ваш вариант: '
```

Рис. 2.14: Программа для подсчета варианта

Запустив программу и указав свой номер студенческого билета, я получил свой вариант для дальнейшей работы. (рис. 2.15).



```
abnikilaeva@fedora:~/work/arch-pc/lab06$ touch variant.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ mousepad variant.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf variant.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_1386 -o variant variant.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./variant
Введите N студенческого билета:
64321234567
Ваш вариант: 4
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.15: Запуск программы для подсчета варианта

2.3. Ответы на контрольные вопросы

1. За вывод сообщения “Ваш вариант” отвечают строки кода:

```
mov eax, rem  
call sprint
```

2. Инструкция mov ecx, x используется, чтобы положить адрес вводимой строки x в регистр ecx mov edx, 80 - запись в регистр edx длины вводимой строки call sread - вызов подпрограммы из внешнего файла, обеспечивающей ввод сообщения с клавиатуры.
3. call atoi используется для вызова подпрограммы из внешнего файла, которая преобразует ascii-код символа в целое число и записывает результат в регистр eax.
4. За вычисления варианта отвечают строки:

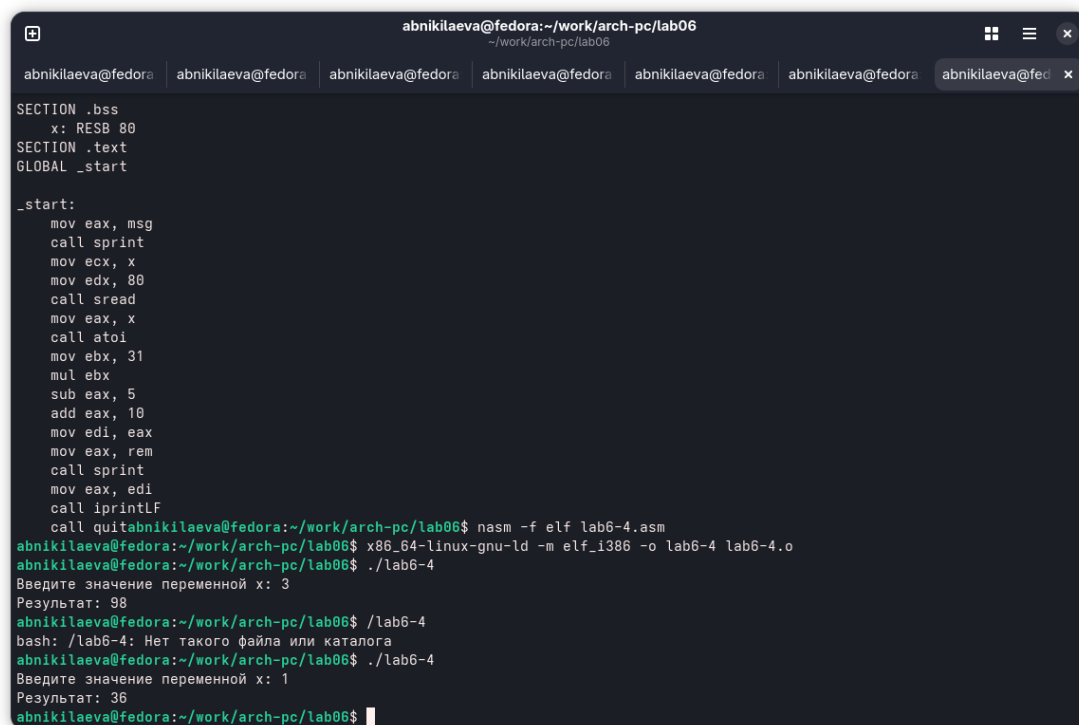
```
xor edx, edx ; обнуление edx для корректной работы div  
mov ebx, 20 ; ebx = 20  
div ebx ; eax = eax/20, edx - остаток от деления  
inc edx ; edx = edx + 1
```

5. При выполнении инструкции div ebx остаток от деления записывается в регистр edx.
6. Инструкция inc edx увеличивает значение регистра edx на 1.
7. За вывод на экран результатов вычислений отвечают строки:

```
mov eax, edx  
call iprintLF
```

2.4. Задание для самостоятельной работы

В соответствии с выбранным вариантом, я реализую программу для подсчета функции $f(x) = 10 + (31x - 5)$, проверка на нескольких переменных показывает корректное выполнение программы (рис. 2.16).



```
abnikilaeva@fedora:~/work/arch-pc/lab06
SECTION .bss
x: RESB 80
SECTION .text
GLOBAL _start

_start:
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    mov ebx, 31
    mul ebx
    sub eax, 5
    add eax, 10
    mov edi, eax
    mov eax, rem
    call sprint
    mov eax, edi
    call iprintLF
    call quitabnikilaeva@fedora:~/work/arch-pc/lab06$ nasm -f elf lab6-4.asm
abnikilaeva@fedora:~/work/arch-pc/lab06$ x86_64-linux-gnu-ld -m elf_i386 -o lab6-4 lab6-4.o
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-4
Введите значение переменной x: 3
Результат: 98
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-4
bash: ./lab6-4: Нет такого файла или каталога
abnikilaeva@fedora:~/work/arch-pc/lab06$ ./lab6-4
Введите значение переменной x: 1
Результат: 36
abnikilaeva@fedora:~/work/arch-pc/lab06$
```

Рис. 2.16: Запуск и проверка программы

Прилагаю код своей программы:

```
%include 'in_out.asm'

SECTION .data
msg: DB 'Введите значение переменной x: ',0
rem: DB 'Результат: ',0

SECTION .bss
x: RESB 80

SECTION .text
GLOBAL _start

_start:
```

```
mov eax, msg
call sprint

mov ecx, x
mov edx, 80
call sread

mov eax, x
call atoi
mov ebx, 31
mul ebx

sub eax, 5
add eax, 10
mov edi, eax
mov eax, rem
call sprint
mov eax, edi
call iprint
```

3. Выводы

При выполнении данной лабораторной работы я освоил арифметические инструкции языка ассемблера NASM.

6. Список литературы

1. Пример выполнения лабораторной работы
2. Курс на ТУИС
3. Лабораторная работа №6
4. Программирование на языке ассемблера NASM Столяров А. В.